

# Treinamento × *inferência*

Todo modelo de IA tem duas vidas: a de **treinamento**, em que aprende, e a de **inferência**, em que responde. Quase toda empresa vive só a segunda, e é nela que ficam o custo recorrente, a latência e a escolha de hardware. Nesta aula: o que acontece em cada fase, um modelo de brinquedo que treina e infere em poucas linhas, e o que isso muda nas decisões de negócio.

Pré-treino

Fine-tuning

Alinhamento

Prefill e decode

Memória da GPU

Custo por token

Formação Claude Code 2026 · Baseado no glossário da Anthropic, no artigo InstructGPT (arXiv:2203.02155), no blog da NVIDIA sobre inferência de LLMs, no cartão do gpt-oss-120b e na página de modelos do Groq, consultados em outubro de 2026 · Exemplo executado em [exemplos/](#)

# Aprender uma vez, *responder milhões de vezes*



|           | TREINAMENTO                                 | INFERÊNCIA                            |
|-----------|---------------------------------------------|---------------------------------------|
| O que faz | Muda os parâmetros do modelo                | Usa os parâmetros sem mudá-los        |
| Quando    | Antes do lançamento (e em ajustes pontuais) | A cada pergunta, o tempo todo         |
| Custo     | Alto, concentrado, de poucos laboratórios   | Recorrente, por uso, de todo mundo    |
| Quem faz  | Fabricante do modelo                        | Você (por API ou em servidor próprio) |

## O pré-treino: *prever a próxima palavra*

- ✓ O modelo recebe um corpus enorme e aprende a **prever a próxima palavra** (na prática, o próximo token) dado o texto anterior
- ✓ Ao fazer isso bilhões de vezes, ele absorve gramática, fatos, estilos e padrões de raciocínio
- ✓ O resultado, sozinho, ainda não é bom em responder perguntas ou seguir instruções (glossário da Anthropic)

**Por que as empresas raramente treinam do zero:** exige volumes imensos de dados, muita capacidade de computação e equipe especializada. Quase todo projeto parte de um modelo já treinado.

O que foi aprendido no treino tem uma **data de corte**: o modelo não sabe o que aconteceu depois (a página da Anthropic informa o corte de conhecimento de cada modelo).

## Fine-tuning e *alinhamento*

### 1 Fine-tuning

Treino adicional com um conjunto menor e específico, para o modelo imitar os padrões desse conjunto (um tom, um formato, um domínio).

### 2 Alinhamento com feedback humano

Pessoas comparam respostas e o modelo é ajustado para seguir melhor a intenção de quem pergunta.

- ✓ No estudo do InstructGPT, avaliadores preferiram as respostas de um modelo de **1,3 bilhão** de parâmetros às do GPT-3 de **175 bilhões**, com 100 vezes menos parâmetros
- ✓ Lição: **mais parâmetros não bastam**; como o modelo é ajustado importa muito
- ✓ As respostas também ficaram mais verdadeiras e menos tóxicas, embora ainda errassem em coisas simples

Fonte: Ouyang et al. (arXiv:2203.02155).

## Um modelo de brinquedo: *treino*

treino\_inferencia.py · parte 1

```
texto = """o gato dorme no sofa . o gato come o peixe .  
o cachorro dorme no tapete . o cachorro come a carne .  
a menina le o livro no sofa . a menina come o bolo ."""  
palavras = texto.split()  
  
# TREINO: percorre o texto uma vez e conta quem vem depois de quem  
modelo = defaultdict(Counter)  
for atual, proxima in zip(palavras, palavras[1:]):  
    modelo[atual][proxima] += 1  
print("palavras no vocabulario:", len(set(palavras)))  
print("depois de 'o':", dict(modelo["o"]))
```

SAÍDA DESTA PARTE

```
palavras no vocabulario: 16  
depois de 'o': {'gato': 2, 'peixe': 1, 'cachorro': 2, 'livro': 1, 'bolo': 1}
```

- ✓ "Treinar" aqui é **contar** quem aparece depois de quem, em um texto de 3 frases
- ✓ Depois de "o", o texto tem "gato" 2 vezes, "cachorro" 2 vezes e outras palavras 1 vez
- ✓ O que foi contado é o **modelo**: o dicionário `modelo`

## O mesmo brinquedo: *inferência*

treino\_inferencia.py · parte 2

```
# INFERENCIA: usa o que foi aprendido, sem mudar nada no modelo
def gerar(inicio, tamanho, semente):
    rnd = random.Random(semente)
    saida = [inicio]
    for _ in range(tamanho):
        opcoes = modelo[saida[-1]]
        if not opcoes:
            break
        saida.append(rnd.choices(list(opcoes),
list(opcoes.values()))[0])
    return " ".join(saida)

for s in (1, 2, 3):
    print(f"geracao {s}:", gerar("o", 6, s))
```

SAÍDA DESTA PARTE

```
geracao 1: o gato come a carne . o
geracao 2: o bolo . o gato come a
geracao 3: o gato come o cachorro come o
```

- ✓ A inferência **não muda o modelo** : só o consulta
- ✓ Gera **uma palavra por vez**, escolhendo a próxima com probabilidade proporcional ao que viu no treino
- ✓ Cada geração sai diferente (a escolha é sorteada), como a temperatura nos LLMs

# O que muda *em um modelo de verdade*

| NO BRINQUEDO                  | NUM LLM                                                          |
|-------------------------------|------------------------------------------------------------------|
| 16 palavras                   | Dezenas ou centenas de milhares de tokens no vocabulário         |
| Olha só a palavra anterior    | Considera todo o contexto: prompt, histórico e o que já gerou    |
| "Aprende" contando            | Aprende ajustando bilhões de parâmetros de uma rede neural       |
| Fica em um dicionário pequeno | Fica em um arquivo de pesos de dezenas a centenas de GB          |
| Não entende nada              | Generaliza padrões, mas continua sendo previsão de próximo token |

A ideia central é a mesma nos dois: **gerar o próximo token e repetir**. Por isso o modelo pode soar seguro e estar errado: ele escolhe a continuação mais provável, não consulta uma base de verdades.

## Duas etapas *em cada resposta*

### PREFILL

processa todos os tokens do prompt de uma vez, em paralelo



### DECODE

gera a resposta, um token por vez

**Efeito prático:** respostas longas demoram porque cada token novo é uma volta do decode. Pedir "em até 3 linhas" (módulo 39) deixa a resposta mais rápida e mais barata.

- ✓ O **prefill** lê o prompt; quanto maior o prompt, mais trabalho aqui
- ✓ O **decode** é sequencial (cada token depende do anterior) e é limitado pela **memória** da GPU, não pela capacidade de cálculo

Fonte: blog técnico da NVIDIA, "Mastering LLM Techniques: Inference Optimization".

# A memória da GPU *manda*

- ✓ Dois itens dominam a memória da GPU numa inferência: os **pesos do modelo** e o **KV cache** (guarda o que já foi calculado do contexto, para não recalculá-lo a cada token)
- ✓ O KV cache cresce linearmente com o tamanho do lote e com o tamanho da sequência: contexto longo e muitos usuários ao mesmo tempo ocupam muita memória
- ✓ A **quantização** guarda os pesos com menos bits, para o modelo caber em menos memória

| GPT-05S-120B (117 BI DE PARÂMETROS) | MEMÓRIA SÓ DOS PESOS* |
|-------------------------------------|-----------------------|
| 16 bits por parâmetro (2 bytes)     | ≈ 234 GB              |
| ~4 bits por parâmetro (0,5 byte)    | ≈ 59 GB               |

\* Estimativa de ordem de grandeza: parâmetros × bytes. O cartão do modelo informa que, com quantização MXFP4, ele roda em uma GPU de 80 GB.

# Por que a saída *custa mais que a entrada*

| MODELO                     | ENTRADA   | SAÍDA     | SAÍDA ÷ ENTRADA |
|----------------------------|-----------|-----------|-----------------|
| <b>gpt-oss-120b (Groq)</b> | US\$ 0,15 | US\$ 0,60 | 4×              |
| <b>Claude Sonnet 5.5</b>   | US\$ 2    | US\$ 10   | 5×              |
| <b>Claude Haiku 4.5</b>    | US\$ 1    | US\$ 5    | 5×              |

US\$ por milhão de tokens, nas páginas do Groq e da Anthropic, outubro de 2026.

- ✓ O prefill processa muitos tokens em paralelo; o decode gera um de cada vez. Os preços de saída maiores costumam refletir essa diferença de trabalho (os provedores não detalham a composição do preço)
- ✓ Na prática: **prompts longos e respostas curtas** custam bem menos que o inverso
- ✓ O Groq roda o gpt-oss-120b a cerca de 500 tokens por segundo, segundo a página de modelos: a velocidade da inferência também depende do hardware

## Adaptar um modelo: *comece pelo mais simples*

### 1 Prompt (módulo 39)

Papel, formato, exemplos. Custa o tempo de testar, não muda o modelo, e resolve boa parte dos casos.

### 2 RAG (aula 09)

Quando falta conhecimento da empresa ou atualizado: busca os trechos e entrega no contexto, sem treinar.

### 3 Fine-tuning

Quando o problema é *comportamento* recorrente (tom, formato, vocabulário) e há muitos exemplos bons. Exige dados, avaliação e manutenção a cada modelo novo.

Fine-tuning **não** é a forma indicada de "ensinar fatos novos" à empresa: para conhecimento que muda, RAG é mais simples de atualizar.

## Teste o que *ficou*

1. Durante a inferência, os parâmetros do modelo:

A) Mudam a cada pergunta

B) Não mudam

C) São apagados

D) São retreinados à noite

B.

2. O decode gera a resposta:

A) Toda de uma vez

B) Um token por vez

C) Só em português

D) Sem usar memória

B.

3. Para o modelo responder com dados atualizados da empresa, o caminho mais simples é:

A) Treinar do zero

B) RAG

C) Aumentar a temperatura

D) Trocar a GPU

B.

4. Por que pedir respostas mais curtas barateia a chamada?

A) A saída tem preço maior por token e é gerada token a token

B) A entrada é gratuita

C) Muda o modelo

D) Não barateia

A.

# Treinar é raro, *inferir é todo dia*

O treinamento ajusta os parâmetros uma vez, em poucos laboratórios; a inferência usa o modelo pronto a cada pergunta, gerando token por token, e é onde a empresa paga em custo, latência e hardware. Antes de pensar em fine-tuning, esgote o prompt e o RAG, e controle o tamanho de entrada e saída.

Pré-treino

Fine-tuning

Prefill / decode

KV cache

Quantização

Prompt → RAG → fine-tuning

Próxima aula → 05 • Como Empresas estão Usando IA Generativa

Fontes: [docs.anthropic.com/en/docs/about-claude/glossary](https://docs.anthropic.com/en/docs/about-claude/glossary); [arxiv.org/abs/2203.02155](https://arxiv.org/abs/2203.02155); [developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization](https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization); [huggingface.co/openai/gpt-oss-120b](https://huggingface.co/openai/gpt-oss-120b); [console.groq.com/docs/models](https://console.groq.com/docs/models); [docs.anthropic.com/en/docs/about-claude/models/overview](https://docs.anthropic.com/en/docs/about-claude/models/overview), consultados em outubro de 2026. O exemplo `treino_inferencia.py` é um modelo didático de bigramas, não um LLM.